SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

(192425222)

I K.Sumanth Kumar Reddy *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

K.Sumanth
Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

① Problem understanding:-

→ The agent starts from a fixed position in a grid world

→ The objective is to read the goal while avoiding obstacles.

Constraint Analysis:-

→ The agent cannot move outside the grid

→ Obstacle cells must be avoided

→ The shortest path should be selected

→ state space All grid positions

→ A chain space: UP, Down, Left, Right

Reward function:-

- Goal = +100
- Obstacele = -100
- Normal move = -1
- Invalid move = -5

Python code:-

```
import random
    stole = (0,0)
    goal = (4,4)
Obstacle = {(1,1),(2,2),(3,1)}

    actions = {
            0 : (-1,0),
            1 : (1,0),
            2 : (0,-1),
            3 : (0,1)}
```

```
for _ in range (2,0):
    action = random.randint(0,3)
    move = actions [action]
new stack =(
        max (0, min (4, state [0]t move [0]))
        Max (0, Min (4, state [1] + move [1]))
    )
if new state in obstacles:
    Print ("obstacle ++; +")
    break
elif new state = =goal;
    Print ("Goal Reached")
    break
    State = new state
    print (state)
```

The Grid world environment consists of states, actions, reward and obstacles, The agent explore the environment by moving in four directions.

The solution satisfies all constraints by preventing invalid moment avoiding obstacles and rewarding successful navigation.

② ε-Greedy Multi-Armed Bandit

Problem understanding:

→ The objective is to maximize reward within 500 trails

→ The e-greedy algorithm balances exploration and exploitation.

Constraint Analysis:-

-> Total trails are limited to 500
-> Higher ε increase exploration
-> lower ε increase exploitation

Python code:-

```
import numpy as np
epsilon=01
rewards = np.random.rand(5)
Q = np.zeros(5)
N = np.zeros(5)
for i in range (500):
    if np random rand( ) epsilon:
        action= np.random.randint(5)
    else
        action= np.argmax(0)
        reward = rewards[action]
        N[action]+=1
        Q[action]+=(reward - Q[action]/N[action]
        print(Q)
```

⇒ ε=0.1 provides the balance because agent exploit the best action while still exploring occassionally

⇒ The ε-greedy algorithm efficiently maximizes/accumulate reward within the 500 -trail constraint)

③ Markov Decision Process for Autonomous Robot

Problem understanding:-

The robot must reach its destination using minimum battles power.

Constraint Analysis:-

→ Battery capacity is limited
→ The robot should avoid unnecessary movement
→ The s Destination must be reached safely.

MDP components:

State:
→ Robot locations

Actions:
- move up
- move Down
- move left
- move Right

Transition Probability:-
- correct more = 0.9
- wrong move = 0.1

Reward:-
- Goal = +100
- Normal move = -1
- Battery over = -5
- collision = -100

Energy constraint:

⇒ The robot should complete navigation before battery depletion

⇒ The reward function minimizes energy usage while encouraging successful navigation

⇒ The MDP enables the robot to learn an energy-efficient policy.

④ Value Iteration Using Python

Problem understanding:

Value iteration computes the optional policy by repeatedly updating state values using the Bellman equation.

Constraint Analysis:

- Restricted states cannot reentered
- The shortest safe path is required
- Goal state provide maximum reward

Python Code:

```
import numpy as np
gamma=0.9
V=np.zeros(5)
for i in range(20):
  for s in range(4):
```

V[s] = max(-1 + gamma x V[s] + [J; V(s])

Print V

V(s) - Value of current state

R = Immediate reward

V = Discount factor

V(s) = Value of next State

Conclusion:-

Value Iteration repeatedly applies the Bellam equation until state values converge ensuring the optimal policy is found avoiding restricted states

(5) RL Framework for Warehouse Robot

Problem understanding:-

The warehouse robot must complete the maximum number of deleveries within a 30-minute battery limit. The RL agent should check efficient routes and avoid unnecessary movement.

Constraint Analysis:-

- Battery life is limited to 30 minute
- Deliveries should be completed before the battery runs out.
- The robot should avoid long or block route.

RL framework:-

- State: Robot location, battery level, Pending deliveries
- Actions: move forward, move Right, move left, Delivery Package, Return to charging station

Reward function:-

Successful Deliveries = +150
Battery saved = +10
Normal move = -1
Battery Exhausted = -100

Outcome:-

The RL agent learns an optimal Policy that:

- Maximizes the number of deliveries
- Minimizes energy usage
- Avoids collisions and battery exhaustion
- Operates withing the 30-minute battery constraint.